

[TÊN TRƯỜNG]
[KHOA / BỘ MÔN]

BÁO CÁO ĐỀ TÀI THỰC HÀNH

TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ TÀI 36

HỆ THỐNG QUẢN LÝ CHUNG CƯ / CĂN HỘ

Flask · PostgreSQL · Nginx · Prometheus · Grafana · Loki

Thông tin	Nội dung
Sinh viên	[HỌ VÀ TÊN]
Mã số sinh viên	[MSSV]
Lớp / khóa	[LỚP / KHÓA]
Giảng viên hướng dẫn	[HỌ TÊN GIẢNG VIÊN]

THÁNG 10 NĂM 2026

TÓM TẮT

Báo cáo trình bày thiết kế và cấu hình một hệ thống quản lý chung cư cho đề tài thực hành số 36. Ứng dụng cung cấp các nghiệp vụ quản lý cư dân, theo dõi phí dịch vụ, phát hành thông báo và tiếp nhận phản ánh. PostgreSQL là cơ sở dữ liệu; pgAdmin hỗ trợ quản trị; Nginx tiếp nhận HTTP và chuyển tiếp yêu cầu đến các dịch vụ bên trong.

Hệ thống quan sát gồm Prometheus thu thập metrics, Grafana hiển thị dashboard, Loki lưu log và Promtail đọc log ứng dụng cùng access log Nginx. Các thành phần được khai báo bằng Docker Compose, phân thành các mạng Docker riêng và chỉ công bố cổng Nginx ra máy host.

Stack đã được chạy cục bộ ngày 08/10/2026. Website, pgAdmin, Grafana và Prometheus truy cập qua Nginx trả HTTP 200; cả 5/5 Prometheus target ở trạng thái UP; dashboard có 8 panel. Ba truy vấn LogQL trả lần lượt 14, 29 và 2 dòng. Các ô ảnh vẫn chờ screenshot giao diện thật trước khi nộp.

Mục tiêu thực hành

- Tổ chức mã nguồn, cấu hình dịch vụ và hướng dẫn chạy trong cùng repository.
- Triển khai ứng dụng có cơ sở dữ liệu PostgreSQL và công cụ pgAdmin.
- Thiết lập reverse proxy Nginx với security headers cơ bản.
- Thu thập metrics container, web server và database; tra cứu log bằng LogQL.
- Áp dụng quyền tối thiểu, mật khẩu mạnh, mạng cô lập và container non-root.

1. PHẠM VI VÀ YÊU CẦU

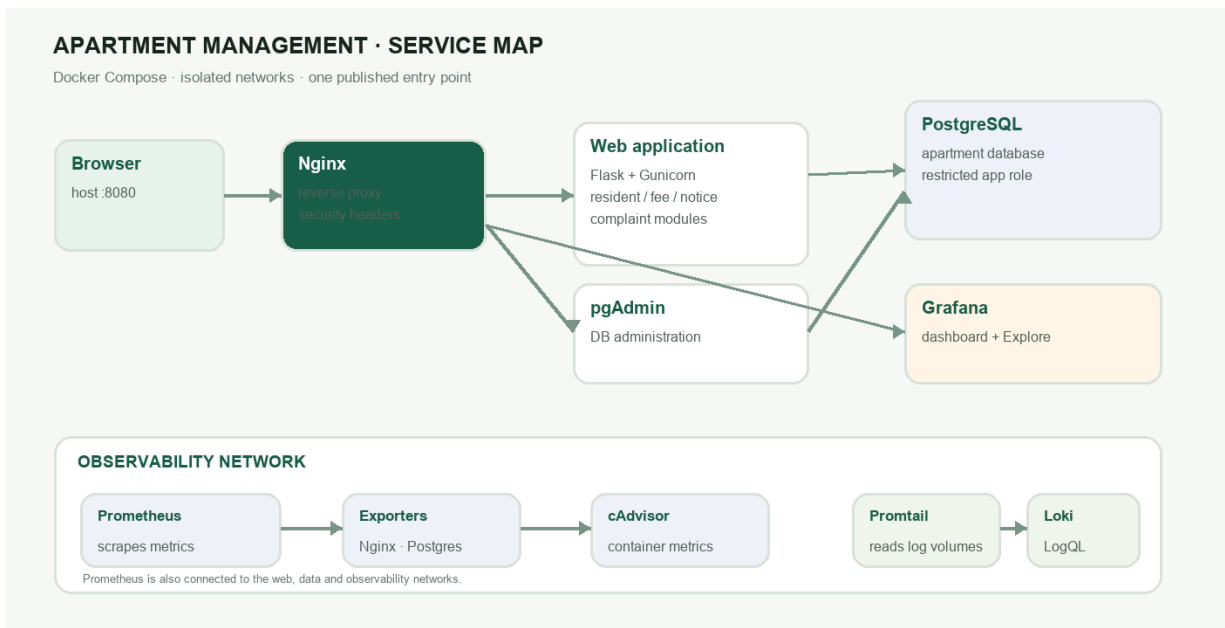
Đề tài được giới hạn ở một tòa nhà mẫu tên An Bình. Dữ liệu mẫu trong ứng dụng là dữ liệu giả phục vụ trình diễn; không nhập thông tin cư dân thật khi đưa repository lên GitHub.

Hạng mục đề	Thiết kế trong dự án	Minh chứng cần ghi nhận
Mã nguồn / GitHub	Repository gồm Flask app, cấu hình Compose, monitoring, logging, README và báo cáo.	URL repository, tài khoản theo MSSV và ba commit rõ nội dung.
Ứng dụng + database	Flask/Gunicorn quản lý cư dân, phí, thông báo, phản ánh; PostgreSQL + pgAdmin.	Trang chức năng và bảng dữ liệu trong pgAdmin.
Reverse proxy	Nginx làm điểm vào duy nhất, định tuyến theo path và thêm security headers.	Truy cập website và response headers.
Giám sát	Prometheus scrape web, Nginx exporter, postgres-exporter, cAdvisor; Grafana provision dashboard.	Targets UP và dashboard có dữ liệu.
Log tập trung	Promtail đọc log JSON từ volume; Loki nhận log; Grafana Explore chạy LogQL.	Ba query LogQL trả về log.
Hardening	Non-root, network isolation, DB role hạn chế, secret file, CSRF, headers, read-only.	Compose, header response và phân tích quyền.

Phạm vi triển khai. Hệ thống dùng HTTP nội bộ với security headers vì đề cho phép lựa chọn này thay cho HTTPS tự ký. Khi triển khai trên mạng thật cần cấu hình TLS, thay mật khẩu demo và rà soát lại CSP theo các ứng dụng được proxy.

2. KIẾN TRÚC VÀ CÁCH HOẠT ĐỘNG

Trình duyệt chỉ kết nối cổng HTTP của Nginx. Nginx chuyển yêu cầu ứng dụng đến Flask/Gunicorn; các path /pgadmin/, /grafana/ và /prometheus/ lần lượt dẫn tới công cụ quản trị hoặc quan sát. PostgreSQL không publish port trực tiếp ra host.



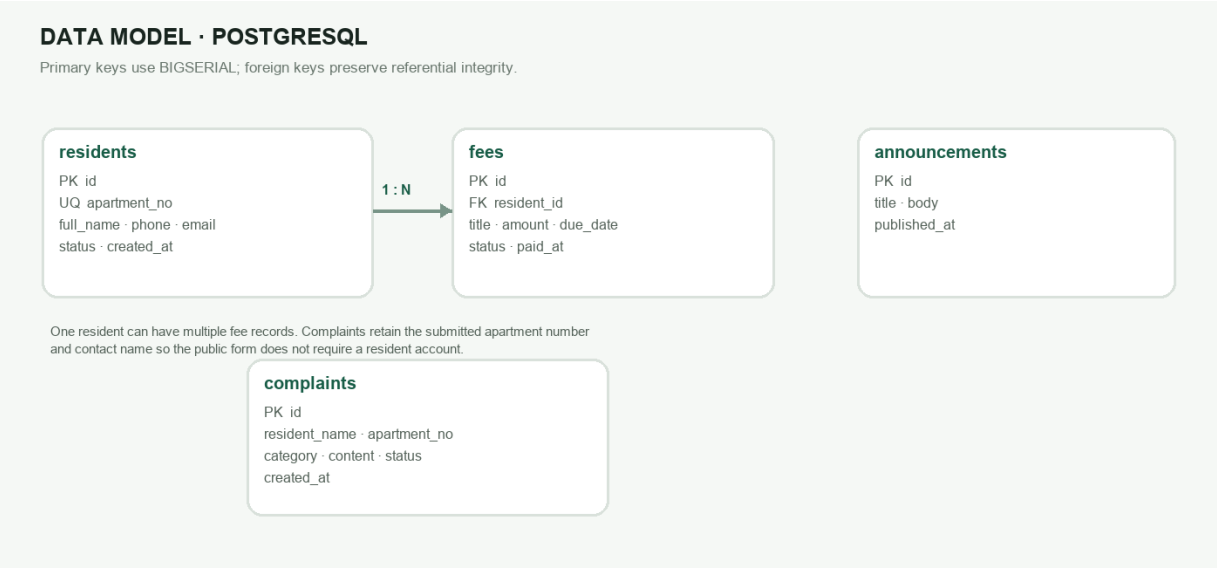
Hình 1. Sơ đồ dịch vụ và các luồng metrics, log của hệ thống.

Bốn mạng Docker định nghĩa ranh giới kết nối. apartment-ingress chỉ gắn với Nginx để publish cổng host; apartment-edge là mạng nội bộ nơi Nginx giao tiếp với web, pgAdmin, Grafana và Prometheus; apartment-data chứa database cùng các client được cấp quyền; apartment-observability nối Prometheus, Grafana, Loki và exporter. Prometheus cần tham gia nhiều mạng để scrape mục tiêu ở các vùng khác nhau.

Dữ liệu bền vững nằm trong named volumes: PostgreSQL, pgAdmin, Grafana, Prometheus, Loki và log. Lệnh docker compose down dừng container nhưng giữ dữ liệu; lệnh xóa volume mới làm mất trạng thái demo.

3. MÔ HÌNH DỮ LIỆU VÀ NGHIỆP VỤ

Cơ sở dữ liệu apartment có bốn nhóm bảng. residents lưu căn hộ và liên hệ; fees lưu từng khoản phải thu; announcements lưu nội dung gửi cư dân; complaints lưu phản ánh, nhóm vấn đề và trạng thái xử lý. Bảng fees tham chiếu residents bằng khóa ngoại resident_id.



Hình 2. Mô hình quan hệ dữ liệu ở mức khái niệm.

Nghệp vụ	Đọc dữ liệu	Ghi dữ liệu
Cư dân	Danh sách theo mã căn hộ.	Thêm hồ sơ mới; mã căn hộ duy nhất.
Phí dịch vụ	Khoản thu, hạn nộp, tổng chưa thu.	Đánh dấu đã thanh toán và ghi paid_at.
Thông báo	Sắp xếp mới nhất trước.	Tạo thông báo từ giao diện quản lý.
Phản ánh	Ban quản lý xem toàn bộ hàng đợi.	Cư dân gửi biểu mẫu; quản lý cập nhật trạng thái.

Các lệnh SQL dùng placeholder tham số thay vì nối chuỗi đầu vào. Biểu mẫu được bảo vệ bằng CSRF token. Dữ liệu mẫu được chèn idempotent ở mức căn hộ và tiêu đề, tránh nhân bản mỗi lần web container khởi động.

4. ỨNG DỤNG VÀ CƠ SỞ DỮ LIỆU

Ứng dụng web

Ứng dụng viết bằng Flask, chạy sau Gunicorn với một worker và bốn thread. Người quản lý đăng nhập bằng mật khẩu được sinh khi tạo .env. Dashboard tổng hợp số cư dân, phí chưa thu, tổng tiền và phản ánh chưa giải quyết. Các trang chức năng cho phép thêm cư dân, ghi nhận thanh toán, đăng thông báo và cập nhật tiến độ phản ánh.

PostgreSQL và pgAdmin

Container PostgreSQL khởi tạo database apartment và tài khoản bootstrap apartment_admin. Script init tạo role apartment_app với NOSUPERUSER, NOCREATEDB, NOCREATEROLE; role này được cấp CONNECT, quyền sử dụng/tạo bảng trong schema public và pg_monitor để exporter thu thập metrics. Ứng dụng và pgAdmin kết nối bằng apartment_app.

Cấu hình	Giá trị	Lý do
Database	apartment	Tên thống nhất.
Application role	apartment_app	Không dùng bootstrap superuser.
Network	apartment-data	Internal, không publish 5432.
Persistence	postgres-data	Giữ dữ liệu qua vòng đời container.
DB tool	pgAdmin qua /pgadmin/	Giao diện quản trị sau Nginx.

Kết quả runtime: container web, PostgreSQL và pgAdmin đều healthy; website /login trả HTTP 200 qua Nginx. Buổi demo vẫn cần thao tác tạo cư dân, ghi nhận khoản phí, đăng thông báo, gửi phản ánh và mở các bảng tương ứng trong pgAdmin; chèn ảnh thật vào khung minh chứng.

5. NGINX REVERSE PROXY

Nginx dùng image unprivileged và lắng nghe cổng container 8080. Cổng host mặc định chỉ bind vào 127.0.0.1:8080; người dùng truy cập website qua một điểm vào. Các service phía sau chỉ expose cổng trên mạng Docker.

Path	Upstream	Mục đích
/	web:8000	Ứng dụng Flask.
/pgadmin/	pgadmin:80	Quản lý PostgreSQL.
/grafana/	grafana:3000	Dashboard metrics và log.
/prometheus/	prometheus:9090	Kiểm tra targets và PromQL.
/healthz	Nginx local	Healthcheck reverse proxy.
/nginx_status	Nginx stub_status	Chỉ exporter trong edge network.

Security headers và giới hạn truy cập

- X-Content-Type-Options: nosniff và X-Frame-Options: DENY.
- Referrer-Policy, Permissions-Policy và Content-Security-Policy được thêm ở Nginx.
- Giới hạn request đăng nhập và gửi phản ánh để giảm spam trong lab.
- Nginx chuyển tiếp Host, địa chỉ client và scheme để Flask nhận đúng thông tin proxy.

Phương án của bài tập là security headers qua HTTP, không phải HTTPS. Không đặt HSTS khi chưa phục vụ TLS. CSP có inline/eval để tương thích Grafana và pgAdmin sau subpath; khi dùng thật cần giới hạn theo kết quả kiểm tra trình duyệt.

[DÁN ẢNH THẬT SAU KHI DEMO: website qua Nginx và response headers]

6. PROMETHEUS VÀ GRAFANA

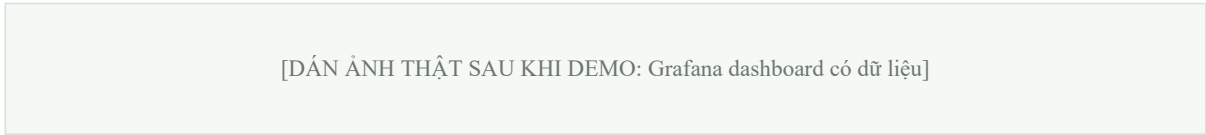
Prometheus scrape mỗi 15 giây. Ứng dụng xuất số request theo method/endpoint/status và histogram latency. Nginx exporter đọc stub_status; postgres-exporter dùng role apartment_app; cAdvisor lấy số liệu CPU và bộ nhớ container từ filesystem host chỉ-đọc.

Job	Target	Metrics minh họa
apartment-web	web:8000/metrics	apartment_http_requests_total, latency histogram
nginx	nginx-exporter:9113	nginx_connections_active
postgres	postgres-exporter:9187	pg_up và chỉ số PostgreSQL
containers	cadvisor:8080	CPU, memory của container
prometheus	localhost:9090	up và trạng thái scrape

Grafana tự nạp datasource Prometheus và Loki cùng dashboard Apartment Management Overview. Dashboard có 8 panel cho request, latency p95, PostgreSQL exporter, kết nối Nginx, CPU/bộ nhớ container và log. Kết quả runtime: datasource Prometheus/Loki đều OK, 5/5 target UP và API dashboard xác nhận 8 panel.

PROMQL

```
up
rate(apartment_http_requests_total[5m])
pg_up
nginx_connections_active
sum by (name) (rate(container_cpu_usage_seconds_total{id!="/" }[5m]))
```



Khi chụp minh chứng, mở Prometheus Targets và ghi trạng thái từng job. Chỉ đánh dấu job UP sau khi có ảnh hoặc kết quả thực tế; sơ đồ cấu hình trong báo cáo không phải ảnh runtime.

7. LOKI, PROMTAIL VÀ LOGQL

Ứng dụng ghi log JSON ra volume app-logs; Nginx ghi access log JSON ra nginx-logs. Promtail mount hai volume chỉ-đọc, trích trường level/event/status/method và gửi log tới Loki qua mạng observability. Grafana truy vấn Loki bằng datasource đã provision.

LogQL	Mục đích
{service="apartment-web"} json level="ERROR"	Tìm lỗi ứng dụng.
{service="apartment-web"} json event="http_request"	Xem request Flask.
{service="nginx"} json status >= 400	Tìm phản hồi HTTP lỗi.

Label được giới hạn ở service, level, event, status và method. URL không được dùng làm label vì số lượng giá trị có thể tăng liên tục. Path vẫn có trong dòng JSON để lọc khi cần.

Kết quả runtime qua Grafana API: cả ba truy vấn trả trạng thái 200; truy vấn access log, request Flask và HTTP lỗi lần lượt trả 14, 29 và 2 dòng trong khoảng 30 phút tại thời điểm ghi nhận.

Promtail được giữ để khớp yêu cầu môn học. Tài liệu Grafana ghi Promtail đã hết vòng đời (EOL) từ ngày 02/03/2026 và không còn được duy trì; hệ thống này phù hợp cho lab theo đề, còn triển khai lâu dài cần chuyển pipeline sang Grafana Alloy. Nguồn: [Grafana Promtail agent documentation](#).

[DÁN ẢNH THẬT SAU KHI DEMO: Grafana Explore trả về ba query LogQL]

8. HARDENING HỆ THỐNG

Các kiểm soát dưới đây được đặt trong Dockerfile, Compose, script database và Nginx. Mục tiêu là giảm quyền của thành phần ứng dụng và giữ dịch vụ dữ liệu không mở trực tiếp ra host.

Kiểm soát	Triển khai	Lưu ý
Non-root	Flask UID 10001; Nginx UID 101; PostgreSQL chạy tiến trình postgres riêng.	cAdvisor cần đọc host metrics.
Network isolation	Ingress chỉ gắn Nginx; edge, data, observability là internal; DB không publish port.	Nginx là cửa vào host.
Least privilege	apartment_app không superuser; exporter dùng quyền đọc metrics.	pg_monitor dùng cho exporter.
Secrets	Mật khẩu ngẫu nhiên trong .env; .gitignore loại khỏi Git.	Không chụp hoặc commit secret.
Container	read_only, tmpfs, cap_drop, no-new-privileges khi tương thích.	Entry point DB/pgAdmin cần tạo volume.
Web security	CSRF, SQL parameters, cookie HttpOnly/SameSite, security headers.	HTTP demo; dùng TLS ngoài lab.
Log access	Promtail đọc volume chỉ-đọc; không mount Docker socket.	cAdvisor đọc host path chỉ-đọc.

Mật khẩu được tạo riêng cho từng máy bằng script. Không dùng tài khoản bootstrap trong ứng dụng; không commit .env. Trước khi public repository, thay ảnh demo có dữ liệu cá nhân bằng dữ liệu giả.

[DÁN ẢNH THẬT SAU KHI DEMO: docker compose ps và cổng Nginx được publish]

9. TRIỂN KHAI VÀ DEMO

Trên Windows, mở PowerShell tại repository, tạo .env một lần rồi build stack. Stack đã chạy cục bộ; các lệnh dưới đây tái tạo demo. Chụp ảnh giao diện thật và hoàn tất các thao tác nghiệp vụ trước khi nộp.

```
POWERSHELL
./scripts/new-secrets.ps1
docker compose up -d --build
docker compose ps
docker compose logs -f web nginx postgres
```

Thứ tự trình bày

- Đăng nhập; mở dashboard và các trang cư dân, phí, thông báo, phản ánh.
- Thêm cư dân, đăng thông báo và gửi phản ánh mẫu.
- Mở pgAdmin, kết nối postgres:5432 bằng apartment_app và xem các bảng.
- Mở Prometheus Targets và dashboard Grafana; giải thích từng exporter.
- Trong Grafana Explore, chạy ba LogQL và phân tích trường, nhấn.
- Mở Compose, chỉ ra published port, bốn mạng và thiết lập quyền.

Ảnh minh chứng cần chèn

[DÁN ẢNH THẬT SAU KHI DEMO: trang tổng quan ứng dụng]
[DÁN ẢNH THẬT SAU KHI DEMO: pgAdmin và database apartment]
[DÁN ẢNH THẬT SAU KHI DEMO: Prometheus Targets và Grafana]

Các vùng trên là chỗ chèn ảnh thật, không phải kết quả runtime. Che email cá nhân và không hiển thị mật khẩu.

10. ĐÁNH GIÁ VÀ KẾT LUẬN

Bảng này liên kết từng tiêu chí chấm điểm với vị trí minh chứng. Trạng thái phản ánh lần chạy cục bộ ngày 08/10/2026; các ô chờ ảnh cần được thay bằng screenshot thật trước khi nộp.

Tiêu chí	Tệp / giao diện	Trạng thái demo
Quản lý mã nguồn (1.5)	GitHub, README, git log.	[CHỜ REPO]
Ứng dụng + DB (1.5)	Website, pgAdmin, dữ liệu.	[HEALTHY; CHỜ ẢNH/CRUD]
Nginx proxy	URL website, response headers.	[HTTP 200; CHỜ ẢNH]
Prometheus + Grafana	Targets và dashboard.	[5/5 UP; 8 PANEL]
Loki + LogQL	Grafana Explore, ba truy vấn.	[3 QUERY TRẢ LOG]
Hardening	Compose, role DB, headers, mạng.	[ĐÃ CẤU HÌNH]
Tổng thể	Stack, báo cáo, phần trình bày.	[CHỜ REPO + ẢNH]

Kết luận

Thiết kế ghép ứng dụng nghiệp vụ với database và stack quan sát trong cùng Docker Compose. Khi bảo vệ, cần giải thích reverse proxy, role apartment_app, bốn mạng Docker, Prometheus scrape, đường đi của log từ volume qua Promtail vào Loki và ý nghĩa của từng LogQL. Bản nộp còn cần repository GitHub, thông tin sinh viên và screenshot minh chứng giao diện thật.

Hạn chế

Tài khoản quản lý có một người dùng demo; chưa có TLS, phân quyền nhiều vai trò, backup tự động hoặc triển khai production. Promtail đã EOL nhưng được dùng để đáp ứng yêu cầu của đề.

Mốc commit

Commit	Nội dung
1	Ứng dụng + PostgreSQL/pgAdmin + Nginx reverse proxy.
2	Prometheus, exporters, Grafana datasource và dashboard.
3	Loki, Promtail, LogQL, hardening và tài liệu.

Tài khoản GitHub đã xác thực: manhbo1000-sketch. Repository apartment-management chưa được tạo; author commit và bìa báo cáo vẫn dùng placeholder cho đến khi có MSSV, họ tên, lớp, trường và giảng viên hướng dẫn.

Tài liệu tham khảo

Docker Compose: docs.docker.com/compose
Prometheus: prometheus.io/docs
Grafana Docker: [Grafana installation documentation](#)
Promtail lifecycle: [Grafana Promtail agent documentation](#)

Ngày lập báo cáo: 08/10/2026. Cập nhật ngày nộp, thông tin cá nhân, URL GitHub và ảnh demo trước khi nộp.